📘 Blueprint — Plataforma de Estudos para Concursos

Codinome: StudyBase (ou escolha outro) **Stack:** Next.js 14 (App Router) · TypeScript · Prisma · MySQL · Vercel · Tailwind CSS

🧭 Visão do Produto

Uma plataforma SaaS onde o administrador cadastra conteúdo estruturado de matérias para concursos (disciplinas, blocos, tópicos, quizzes, flashcards) e o aluno acessa uma experiência de estudo interativa — igual à página gerada para Atualidades do Mercado Financeiro, mas dinâmica, persistida em banco e escalável para qualquer matéria.

🗂️ Entidades Principais (Domínio)

```
Materia
  └── Bloco (ex: "Bloco 1 — Nova Infraestrutura")
        └── Topico (ex: "Pix — Arquitetura e Modalidades")
              └── TopicoParagrafo (corpo do tópico)
              └── TopicoKeyPoint (destaque em azul)
              └── TopicoListItem (itens de lista)
  └── Quiz
        └── Questao
              └── Alternativa (4 opções, 1 correta)
              └── Explicacao
  └── Flashcard (pergunta + resposta)
  └── ErroIniciante (título + descrição)
  └── Foco (itens "focar" e "ignorar")
  └── Recurso (fonte, link, tipo: livro/site/video)
```

🗃️ Schema Prisma (MySQL)

```prisma
```

```
model Materia {
  id          Int       @id @default(autoincrement())
  slug        String    @unique
  titulo      String
  subtitulo   String?
  descricao   String    @db.Text
  badge       String?
  horasTotal  Int       @default(20)
  ativa       Boolean   @default(true)
  ordem       Int       @default(0)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  blocos          Bloco[]
  flashcards      Flashcard[]
  errosIniciantes ErroIniciante[]
  focos           Foco[]
  recursos        Recurso[]
  quizzes         Questao[]
}

model Bloco {
  id        Int      @id @default(autoincrement())
  materiaId Int
  titulo    String
  subtitulo String?
  emoji     String   @default("🟩🟥")
  horas     Int      @default(4)
  cor       String   @default("#00d4ff")
  ordem     Int      @default(0)

  materia  Materia  @relation(fields: [materiaId], references: [id], onDelete: Casca
  topicos  Topico[]
}

model Topico {
  id         Int    @id @default(autoincrement())
  blocoId    Int
  titulo     String
  frequencia String @default("med") // hot | med | ok
  ordem      Int    @default(0)

  bloco      Bloco            @relation(fields: [blocoId], references: [id], onDelet
  paragrafos TopicoParagrafo[]
  keyPoints  TopicoKeyPoint[]
  listItems  TopicoListItem[]
```

```
}

model TopicoParagrafo {
  id       Int    @id @default(autoincrement())
  topicoId Int
  texto    String @db.Text
  ordem    Int    @default(0)
  topico   Topico @relation(fields: [topicoId], references: [id], onDelete: Cascade)
}

model TopicoKeyPoint {
  id       Int    @id @default(autoincrement())
  topicoId Int
  texto    String @db.Text
  topico   Topico @relation(fields: [topicoId], references: [id], onDelete: Cascade)
}

model TopicoListItem {
  id       Int    @id @default(autoincrement())
  topicoId Int
  label    String?
  texto    String @db.Text
  ordem    Int    @default(0)
  topico   Topico @relation(fields: [topicoId], references: [id], onDelete: Cascade)
}

model Questao {
  id          Int       @id @default(autoincrement())
  materiaId   Int
  enunciado   String    @db.Text
  explicacao  String    @db.Text
  ordem       Int       @default(0)
  materia     Materia   @relation(fields: [materiaId], references: [id], onDelete: C
  alternativas Alternativa[]
}

model Alternativa {
  id        Int     @id @default(autoincrement())
  questaoId Int
  texto     String  @db.Text
  correta   Boolean @default(false)
  ordem     Int     @default(0)
  questao   Questao @relation(fields: [questaoId], references: [id], onDelete: Casca
}

model Flashcard {
  id        Int     @id @default(autoincrement())
```

```
  materiaId Int
  pergunta  String  @db.Text
  resposta  String  @db.Text
  ordem     Int     @default(0)
  materia   Materia @relation(fields: [materiaId], references: [id], onDelete: Casca
}

model ErroIniciante {
  id        Int     @id @default(autoincrement())
  materiaId Int
  titulo    String
  descricao String  @db.Text
  ordem     Int     @default(0)
  materia   Materia @relation(fields: [materiaId], references: [id], onDelete: Casca
}

model Foco {
  id        Int     @id @default(autoincrement())
  materiaId Int
  texto     String
  tipo      String  @default("focar") // focar | ignorar
  ordem     Int     @default(0)
  materia   Materia @relation(fields: [materiaId], references: [id], onDelete: Casca
}

model Recurso {
  id        Int     @id @default(autoincrement())
  materiaId Int
  nome      String
  descricao String?
  tipo      String  @default("livro") // livro | site | video | questoes | atualizac
  url       String?
  prioridade String @default("recomendado") // obrigatorio | recomendado | opcional
  ordem     Int     @default(0)
  materia   Materia @relation(fields: [materiaId], references: [id], onDelete: Casca
}
```

📁 Estrutura de Pastas

```
studybase/
├── app/
|   ├── (admin)/
|   |   ├── layout.tsx              # Layout do painel admin
```

```
│   │   ├── dashboard/page.tsx      # Visão geral
│   │   ├── materias/
│   │   │   ├── page.tsx            # Lista de matérias
│   │   │   ├── nova/page.tsx       # Criar matéria
│   │   │   └── [id]/
│   │   │       ├── page.tsx        # Editar matéria
│   │   │       ├── blocos/page.tsx # Gerenciar blocos
│   │   │       ├── quiz/page.tsx   # Gerenciar quiz
│   │   │       ├── flashcards/page.tsx
│   │   │       ├── focos/page.tsx
│   │   │       └── recursos/page.tsx
│   │   └── context/page.tsx        # 🔑 Gerador de contexto Claude
│   │
│   ├── (public)/
│   │   ├── page.tsx                # Landing page
│   │   └── estudo/
│   │       └── [slug]/page.tsx     # Página de estudo (igual ao HTML gerado)
│   │
│   └── api/
│       ├── materias/route.ts
│       ├── materias/[id]/route.ts
│       ├── blocos/route.ts
│       ├── topicos/route.ts
│       ├── questoes/route.ts
│       ├── flashcards/route.ts
│       └── context/route.ts        # 🔑 API do contexto Claude
│
├── components/
│   ├── admin/
│   │   ├── MateriaForm.tsx
│   │   ├── BlocoForm.tsx
│   │   ├── TopicoEditor.tsx        # Editor rico para tópicos
│   │   ├── QuestaoForm.tsx
│   │   ├── FlashcardForm.tsx
│   │   └── DragList.tsx            # Reordenação drag-and-drop
│   └── estudo/
│       ├── HeroEstudo.tsx
│       ├── BlocoCard.tsx
│       ├── TopicoExpandivel.tsx
│       ├── QuizInterativo.tsx
│       ├── FlashcardDeck.tsx
│       ├── PainelFoco.tsx
│       └── BarrasProgresso.tsx
│
├── lib/
│   ├── prisma.ts
│   ├── context-generator.ts        # 🔑 Gera snapshot do contexto
```

```
│   └── utils.ts
│
└── claude-context/
    └── CONTEXT.md                  # 🔑 Arquivo de contexto vivo
```

🔑 Sistema de Contexto Claude (Funcionalidade Central)

O problema

Ao usar Claude em projetos longos, os tokens acabam e o contexto se perde. É preciso reintroduzir o projeto toda vez.

A solução: Gerador de Contexto Automático

Uma rota `/admin/context` que gera um snapshot completo do estado atual do projeto em formato otimizado para colar em um novo chat Claude.

O que o snapshot inclui:

```markdown
```

```
# CONTEXTO DO PROJETO StudyBase
**Gerado em:** 2025-01-15 14:32

## Stack
Next.js 14 (App Router) · TypeScript · Prisma · MySQL · Tailwind · Vercel

## Estado atual do banco
- 3 matérias cadastradas
- 15 blocos, 48 tópicos, 87 questões, 36 flashcards

## Matérias existentes
| slug | título | blocos | status |
|---|---|---|---|
| atualidades-mercado-financeiro | Atualidades... | 5 | ativa |

## Arquivos críticos modificados recentemente
[lista automática dos últimos arquivos editados]

## Padrões do projeto
- Componentes: PascalCase
- APIs: REST em /api/*
- Estilos: Tailwind + CSS vars do tema
- Tema: dark, emerald/teal gradient

## Último problema trabalhado
[campo editável pelo admin]

## Próximos passos
[campo editável pelo admin]
```

Implementação:

```ts
// app/api/context/route.ts
export async function GET() {
  const materias = await prisma.materia.findMany({
    include: { blocos: { include: { topicos: true } }, _count: true }
  });
  const snapshot = generateContextMd(materias);
  return new Response(snapshot, {
    headers: { 'Content-Type': 'text/markdown' }
  });
}
```

🛠️ Funcionalidades do Admin (MVP)

Fase 1 — Conteúdo (prioridade máxima)

- [ ] CRUD de Matérias (slug, título, descrição, horas)
- [ ] CRUD de Blocos por matéria (com emoji, cor, horas)
- [ ] Editor de Tópicos (parágrafos, key points, lista)
- [ ] Reordenação drag-and-drop de blocos e tópicos
- [ ] CRUD de Questões com 4 alternativas
- [ ] CRUD de Flashcards
- [ ] CRUD de Erros de Iniciante
- [ ] CRUD de Focos (focar/ignorar)
- [ ] CRUD de Recursos

Fase 2 — Experiência do Aluno

- [ ] Página pública `/estudo/[slug]` (renderiza igual ao HTML)
- [ ] Progresso salvo no localStorage por slug
- [ ] Timer de estudo por sessão

Fase 3 — SaaS / Monetização

- [ ] Autenticação (NextAuth ou Clerk)
- [ ] Planos (gratuito com X matérias, pago ilimitado)
- [ ] Dashboard do aluno com histórico de quizzes
- [ ] Compartilhamento de matérias

🚀 Setup Inicial (ordem de execução)

```bash
```

```
# 1. Criar projeto
npx create-next-app@latest studybase --typescript --tailwind --app

# 2. Instalar dependências
npm install prisma @prisma/client
npm install @dnd-kit/core @dnd-kit/sortable   # drag-and-drop
npm install lucide-react                       # ícones

# 3. Configurar Prisma
npx prisma init --datasource-provider mysql
# Colar schema acima em prisma/schema.prisma
npx prisma migrate dev --name init
npx prisma generate

# 4. Variáveis de ambiente (.env)
DATABASE_URL="mysql://user:pass@host:3306/studybase"
NEXTAUTH_SECRET="..."

# 5. Deploy
# Conectar repo ao Vercel
# Adicionar DATABASE_URL nas env vars da Vercel
# Usar PlanetScale ou Railway para MySQL gerenciado
```

💡 Recomendação de MySQL Gerenciado

Opção	Preço	Compatível Vercel	Recomendação
PlanetScale	Grátis até 5GB	✅	⭐ Melhor opção
Railway	$5/mês	✅	Boa alternativa
Neon (Postgres)	Grátis	✅	Se mudar para PG
Supabase (Postgres)	Grátis	✅	Feature-rich

> **Sugestão:** considere migrar para **PostgreSQL + Neon** — é gratuito, tem melhor suporte no ecossistema Next.js e o Prisma abstrai a diferença. MySQL também funciona perfeitamente.

📋 Primeiro Sprint (o que construir primeiro)

1. Setup do projeto + Prisma + banco

2. Página admin de Matérias (listar, criar, editar)
3. Página admin de Blocos e Tópicos
4. Seed com a matéria "Atualidades do Mercado Financeiro" (já temos o conteúdo)
5. Página pública `/estudo/[slug]` renderizando o conteúdo do banco
6. Gerador de contexto Claude

Resultado do Sprint 1: a página de AMF que geramos estará viva, servida do banco, e você poderá editar qualquer conteúdo pelo admin.